inputs as this would have no meaning. As you would expect in the first case the uploaded file is in `params[:picture]` and in the second case in `params[:person][:picture]`.

5.1 What Gets Uploaded

The object in the `params` hash is an instance of a subclass of `IO`. Depending on the size of the uploaded file it may in fact be a `StringIO` or an instance of `File` backed by a temporary file. In both cases the object will have an `original_filename` attribute containing the name the file had on the user's computer and a `content_type` attribute containing the MIME type of the uploaded file. The following snippet saves the uploaded content in `#{Rails.root}/public/uploads` under the same name as the original file (assuming the form was the one in the previous example).

```
def upload
  uploaded_io = params[:person][:picture]
  File.open(Rails.root.join('public', 'uploads',
uploaded_io.original_filename), 'wb') do |file|
    file.write(uploaded_io.read)
  end
end
```

Once a file has been uploaded, there are a multitude of potential tasks, ranging from where to store the files (on disk, Amazon S3, etc) and associating them with models to resizing image files and generating thumbnails. The intricacies of this are beyond the scope of this guide, but there are several libraries designed to assist with these. Two of the better known ones are CarrierWave andPaperclip.
If the user has not selected a file the corresponding parameter will be an empty string.

5.2 Dealing with Ajax

Unlike other forms making an asynchronous file upload form is not as simple as providing `form_for`with `remote: true`. With an Ajax form the serialization is done by JavaScript running inside the browser and since JavaScript cannot read files from your hard drive the file cannot be uploaded. The most common workaround is to use an invisible iframe that serves as the target for the form submission.

6 Customizing Form Builders

As mentioned previously the object yielded by `form_for` and `fields_for` is an instance of `FormBuilder` (or a subclass thereof). Form builders encapsulate the notion of displaying form elements for a single object. While you can of course write helpers for your forms in the usual way, you can also subclass `FormBuilder` and add the helpers there. For example:

```
<%= form_for @person do |f| %>
  <%= text_field_with_label f, :first_name %>
<% end %>
```

can be replaced with

```
<%= form_for @person, builder: LabellingFormBuilder do |f| %>
  <%= f.text_field :first_name %>
<% end %>
```

by defining a `LabellingFormBuilder` class similar to the following:

```
class LabellingFormBuilder < ActionView::Helpers::FormBuilder
  def text_field(attribute, options={})
    label(attribute) + super
  end
end
```

If you reuse this frequently you could define a `labeled_form_for` helper that automatically applies the `builder: LabellingFormBuilder` option: